

PROJECT PROPOSAL

OMNIDROP SIMULATOR

A High-Performance Java Arcade Experience

Java Native

AWT & Swing

Custom Physics Engine

The Vision & Motivation

Why build an engine from scratch? To master core programming logic, memory management, and rendering without relying on external black-box game engines.

THE CORE GAMEPLAY CONCEPT

OmniDrop is a fast-paced, physics-driven arcade game designed to test reflexes and decision-making.

-  **The Guardian:** The player controls a dynamic basket to catch beneficial falling entities.
-  **Dynamic Spawning:** Entities spawn with calculated randomization, featuring varying speeds, wobble physics, and drag.
-  **Difficulty Curve:** As the score increases, fall speeds scale linearly while spawn intervals decrease exponentially.
-  **Combo System:** Sequential catches multiply score rewards, heavily incentivizing risk-taking.



PRIMARY TECHNICAL OBJECTIVES



Fixed-Timestep Loop

Implement a precise Swing Timer operating at 60 FPS (16.6ms). This ensures consistent physics calculations (gravity, drag) independent of hardware rendering speed.



Custom FX Engine

Build a proprietary ParticleSystem from the ground up. Frame-based lifetimes and alpha-scaling will handle dynamic hit sparks and visual popups natively.



Data Persistence

Develop a robust File I/O manager to save high scores directly to the user's home directory, maintaining competitive state across multiple game sessions safely.

PROPOSED TECHNOLOGY STACK

Java (AWT & Swing)

Rationale: Utilizing standard Java libraries ensures cross-platform compatibility and rapid deployment without heavyweight dependencies.

Graphics2D: Will be leveraged for advanced rendering techniques, including RadialGradients, Alpha Compositing, and geometric polygon drawing.

Zero Game Engines

Rationale: By explicitly avoiding tools like Unity or Godot, this project serves as a deep-dive into software architecture.

Learning Outcome: Demonstrates mastery over collision detection algorithms (AABB), event-driven programming, state management, and memory optimization.

ENTITY ARCHITECTURE PLAN

Entity Class	Inheritance	Behavioral Role	Game Impact
Spawn	FallingObject	Primary Catch Target	+10 to +50 Points (Based on Rarity)
Voidling	FallingObject	Hazard to Avoid	-1 Player Life on Collision
PowerUp	FallingObject	Temporary Buff	Grants Shield, Magnet, or 2x Score
Player	Standalone	Input Controller	Defines Catch Radius & Boundaries

| PERFORMANCE TARGET

60

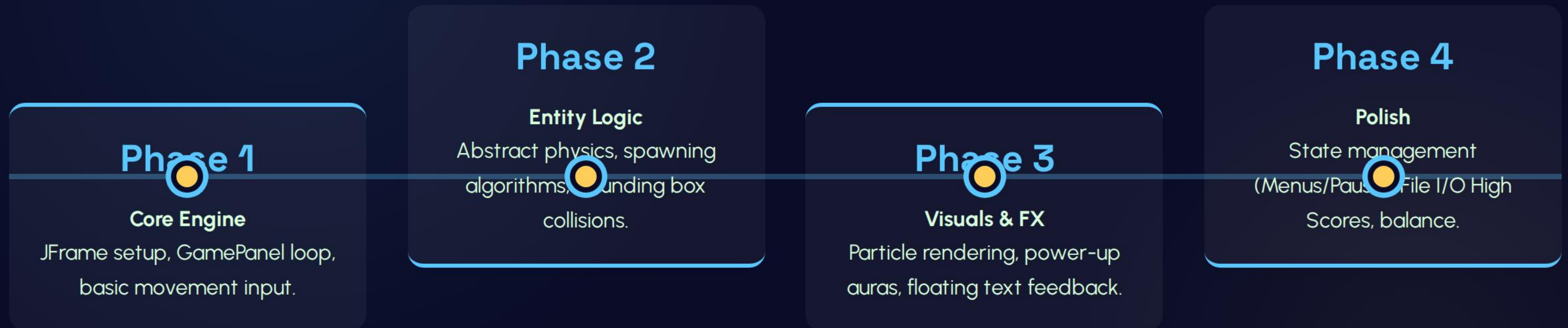
FRAMES PER SECOND

Smooth Execution is Critical

In an arcade simulator, input latency and visual stutter destroy the user experience. The engine is architected to separate the logical update loop from the paint component.

By enforcing a strict 16ms delay threshold, physics transformations (like sine-wave wobble and acceleration) remain perfectly fluid.

DEVELOPMENT ROADMAP



| PROJECT DEVELOPER



Talha Yaseen

AI/ML Enthusiast | BSIT Batch '24

Punjab University Lahore, FCIT

 Talha-Yaseen-Hub/Omni-Drop

 Talha Yaseen | LinkedIn

 talhavectorarts@gmail.com

PROPOSAL CONCLUSION

Are there any questions regarding the architecture or timeline?

 Review Project Repository

IMAGE SOURCES



https://png.pngtree.com/thumb_back/fw800/background/20260101/pngtree-retro-arcade-with-neon-lights-glowing-game-machines-lining-glossy-tiled-image_20951965.webp

Source: [pngtree.com](https://png.pngtree.com)



https://img.magnific.com/premium-photo/hacker-wearing-mask-with-digital-effects-dark-background-concept-hacker-mask-digital_660230-183551.jpg?semt=ais_hybrid&w=740&q=80

Source: www.magnific.com